

QuantumShield™ REST API — NIST-Certified Post-Quantum Cryptography for Secure Cloud Communication

1. Overview

Developers can incorporate **digital signature algorithms, key encapsulation, and quantum-resistant encryption** into their applications using the **Post-Quantum Cryptography REST API (PQC API)**.

Traditional asymmetric algorithms (**RSA, ECC**) are susceptible to **Shor's algorithm-based attacks** as quantum computing develops.

This API provides **hybrid post-quantum cryptographic primitives** that ensure **forward secrecy, algorithmic agility, and quantum-safe key exchange** within existing HTTP-based infrastructures.

2. Authentication

All endpoints use **Quantum-Safe Mutual Authentication (QSMA)**.

Supported Auth Mechanisms

Method	Description	Algorithm
X-PQC-Auth	Custom HMAC-based token	CRYSTALS-Dilithium Signatures
Bearer Token	Quantum-hardened OAuth 2.1	Kyber768-based key exchange
mTLS (Quantum Hybrid)	TLS 1.3 hybrid handshake	X25519 + Kyber512

Note: All credentials are rotated every 12 hours via the /auth/refresh endpoint to maintain PQC integrity and forward secrecy.

3. API Endpoints

3.1. Generate Quantum-Safe Key Pair

POST /api/v1/keys/generate

Description

Generates a **lattice-based key pair** using a NIST-approved post-quantum algorithm (Kyber or Saber).

Request

```
{  
  "algorithm": "Kyber768",  
  "usage": "encapsulation",  
  "format": "base64"  
}
```

Response

```
{  
  "publicKey": "MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQE...",  
  "privateKey": "MIIEowIBAAKCAQEArDE...",  
  "algorithm": "Kyber768",  
  "created_at": "2025-10-28T00:42:00Z"  
}
```

Status Codes

Code Description

- | | |
|-----|------------------------------------|
| 201 | Key pair successfully generated |
| 400 | Invalid algorithm selection |
| 503 | Quantum Entropy Source Unavailable |

3.2. Encrypt Data with Quantum-Safe Public Key

POST /api/v1/encrypt

Description

Encrypts plaintext using **Kyber encapsulation** with AES-GCM hybrid encryption for performance and backward compatibility.

Request

```
{  
  "publicKey": "MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...",  
  "data": "The quick brown fox jumps over the lazy dog"  
}
```

Response

```
{  
  "ciphertext": "aG91c2Ugb2YgcXVhbnR1bSBzYWZlIGNpcGhlcnRleHQ=",  
  "algorithm": "Kyber768-AESGCM",  
  "entropy_source": "QRNG",  
  "timestamp": "2025-10-28T01:00:00Z"  
}
```

3.3. Verify Post-Quantum Digital Signature

POST /api/v1/signature/verify

Description

Verifies a **CRYSTALS-Dilithium** signature for integrity and non-repudiation under quantum-resistant assumptions.

Request

```
{  
  "message": "Quantum safe communication initiated.",  
  "signature": "QmFzZTY0RW5jb2RlZFNpZ25hdHVyZQ==",  
  "publicKey": "MIGJAoGBALsY9KY="
```

Response

```
{  
  "isValid": true,  
  "algorithm": "Dilithium3",  
  "verification_latency_ms": 4.12
```

4. Error Handling

Error Code	Description	Resolution
401	Invalid PQC authentication token	Rotate session using /auth/refresh
422	Unsupported PQC Algorithm	Use NIST-approved set: Kyber, Dilithium, Falcon
500	Entropy Source Fault	Switch to fallback PRNG or hybrid RNG mode
530	Quantum Entropy Pool Exhausted	Wait 120 ms before reattempting

5. Security Model

This API integrates with **Quantum-Resistant Key Management Service (QKMS)** and **Hybrid PQ-TLS 1.3** layers, ensuring:

- **Hybrid Handshake:** Combines classical X25519 with Kyber768 for forward secrecy.
 - **Quantum Randomness Source:** Utilizes **QRNG (Quantum Random Number Generator)** hardware entropy for non-deterministic key material.
 - **Algorithm Agility:** Auto-negotiates secure algorithms via the /capabilities endpoint.
 - **Tamper Detection:** HMAC + Dilithium hybrid MAC verification.
 - **Zero-Knowledge Key Storage:** Private keys encrypted under lattice-based homomorphic shield (experimental).
-

6. Compliance & Certification

Standard	Description
NIST PQC Finalist Compliance	Kyber, Dilithium algorithms (NIST Round 3 finalists)
FIPS 140-3 Level 3	Cryptographic Module Validation
ISO/IEC 27001	Information Security Management
GDPR & CCPA Ready	Privacy-preserving cryptography at rest and in transit

7. Example Integration (Python SDK)

```
import pqc_sdk

client = pqc_sdk.Client(api_key="your-pqc-token")

keypair = client.keys.generate(algorithm="Kyber768")

ciphertext = client.encrypt(public_key=keypair["publicKey"], data="Hello Quantum World!")
```

```
valid = client.signature.verify(  
    message="Hello Quantum World!",  
    signature="QmFzZTY0...",  
    public_key=keypair["publicKey"]  
)
```

8. Best Practices

- Use **Hybrid PQC + Classical** algorithms during the transition phase (2025–2030).
 - Enable **ephemeral key exchange** for every session to resist *harvest-now-decrypt-later* attacks.
 - Periodically verify your endpoint compliance via `/api/v1/health/pqc-integrity`.
 - Store no raw key material—always use **Quantum Hardware Security Modules (QHSM)**.
 - Monitor entropy health using `/metrics/entropy` for randomness assurance.
-

9. Glossary

Term	Description
Kyber	Lattice-based KEM (Key Encapsulation Mechanism) finalist in NIST PQC standardization
Dilithium	Lattice-based signature scheme for quantum-safe authentication
Falcon	Compact lattice-based signature algorithm
QRNG	Quantum Random Number Generator
Hybrid PQC	Combined use of classical and PQ algorithms for transitional security
QKMS	Quantum Key Management System

10. Contact & Support

For enterprise PQC API integration support, contact:

`security@quantumapi.dev`

PGP Fingerprint: AB12 CDEF 3456 7890 1234 5678 9ABC DEF0 1234 5678